

Reviewer Pack — Minimal Symplectic Leaves and Communication Rank Growth Corr...

Entry cf699491a30a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 04:42:50 UTC

Minimal Symplectic Leaves and Communication Rank Growth Correlation

Entry ID: cf699491a30a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 04:42:50 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every planar graph G , the minimal number of symplectic leaves ($\text{msl}(G)$) of its associated moment map is linearly correlated with its communication rank growth ($\text{cr}(G)$), such that $\text{msl}(G) = \Theta(\text{cr}(G))$.

Rationale (proposer's reasoning):

Symplectic geometry has recently been applied to study quantum systems and their complexity, and the concept of symplectic leaves could potentially capture the geometric structure underlying communication protocols. If this structural information is indeed correlated with communication complexity, it may reveal new insights into the design of efficient algorithms.

Taxonomy category: symplectic_geometry_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 639a472b99ba2b9d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\text{msl}(G)$ and $\text{cr}(G)$ across all planar graphs G is greater than or equal to 0.7, with a p-value less than 0.01.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic geometry" AND "communication complexity" AND minimal symplectic leaves" - "moment map" AND "communication rank growth" AND planar graph" - "linear correlation" AND $\text{msl}(G) = \theta(\text{cr}(G))$ AND associated moment map

Top relevant hits considered: - [http://arxiv.org/abs/1107.4977v2] Gamma-Ray and Parsec-Scale Jet Properties of a Complete Sample of Blazars From the MOJAVE Program - [http://arxiv.org/abs/1610.03311v1] Characterizing the gamma-ray long-term variability of PKS 2155-304 with H.E.S.S. and Fermi-LAT - [http://arxiv.org/abs/2409.10306v1] Identifying activity induced RV periodicities and correlations using Central Line Moments

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def generate_planar_graphs(n=40):
    # Placeholder function to generate planar graphs
    # This is a dummy implementation and should be replaced with actual graph generation logic
    return [random.randint(1, n) for _ in range(30)]

def construct_symplectic_leaves(graph):
    if isinstance(graph, list):
        return random.randint(1, len(graph))
    else:
        raise ValueError("Graph must be a list")

def calculate_communication_rank_growth(graph):
    # Placeholder function to calculate communication rank growth
    # This is a dummy implementation and should be replaced with actual algorithm logic
    return random.random()

def run_trial(seed: int) -> dict:
    random.seed(seed)

    instances = generate_planar_graphs(n=40)
    msl_values = [construct_symplectic_leaves(graph) for graph in instances]
    cr_values = [calculate_communication_rank_growth(graph) for graph in instances]

    if len(msl_values) != len(cr_values):
        return {
            "metric_name": "Pearson correlation",
```

```

        "metric_value": None,
        "instances_tested": len(instances),
        "n_max": 40,
        "conjecture_holds": False,
        "counterexample": "Mismatch in instance counts"
    }

pearson_correlation = sum((msl - msl_avg) * (cr - cr_avg) for msl, cr in zip(msl_values, cr_values))
msl_avg = sum(msl_values) / len(msl_values)
cr_avg = sum(cr_values) / len(cr_values)

return {
    "metric_name": "Pearson correlation",
    "metric_value": pearson_correlation,
    "instances_tested": len(instances),
    "n_max": 40,
    "conjecture_holds": pearson_correlation >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Pearson correlation < 0.7' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5c8299fa.py", line 70, in <module>
    result = run_trial(seed)
            ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5c8299fa.py", line 38, in run_trial
    msl_values = [construct_symplectic_leaves(graph) for graph in instances]
                  ~~~~~^~~~~~

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5c8299fa.py", line 27, in construct_sym
    raise ValueError("Graph must be a list")
ValueError: Graph must be a list
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the Pearson correlation coefficient and p-value could not be calculated to verify the conjecture. | next: Investigate the error in the test code and run it again with a corrected input to calculate the Pearson correlation coefficient and p-value.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17145
2	propose	ollama_remote	glm4:latest	0	0	17413
3	propose	ollama_remote	glm4:latest	0	0	19814
4	preregistration	ollama_remote	glm4:latest	0	0	20407
5	novelty	ollama_remote	glm4:latest	0	0	9979
6	novelty	ollama_remote	glm4:latest	0	0	14166
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15605
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10047
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14316
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8559
11	judge	ollama_remote	glm4:latest	0	0	22064

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 169515 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/cf699491a30a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cf699491a30a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cf699491a30a.tar.gz` (if generated)